Compresr SDK · Python

LiteLLM Proxy Guardrail

Drop-in LiteLLM guardrail that shrinks prompts, tool outputs, and chat history before they hit the model — cutting token spend and tail latency with zero application code changes.

1 What you get

Plug Compresr into your existing LiteLLM proxy and every chat-completion request is rewritten on the way out: bulky tool outputs and RAG dumps are query-aware compressed, your user's last message is preserved verbatim, and images, audio, and assistant turns pass through untouched. You keep the same proxy URL, same client SDKs, same upstream model. Customers typically see **40–80% token reduction** on tool-heavy and long-context workloads, which translates directly into lower bills and faster responses.

Note

Every compressed call returns a `compresr_stats` block (messages compressed, tokens saved, ratio, duration) and stamps the `x-litellm-applied-guardrails` response header. Savings are measurable on day one — no extra dashboards required.

2 Install & start the proxy

Step 1 — install the SDK. The `[litellm]` extra brings in `litellm[proxy]`. You'll also need credentials for at least one upstream provider (e.g. `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`).

```
pip install 'compresr[litellm]'
export COMPRESR_API_KEY=cmp_...
```

Step 2 — save your proxy config as `proxy_config.yaml`:

```
model_list:
  - model_name: gpt-5-mini
    litellm_params:
      model: openai/gpt-5-mini
      api_key: os.environ/OPENAI_API_KEY

guardrails:
  - guardrail_name: "compresr"
    litellm_params:
      guardrail: compresr
      mode: pre_call
      api_key: os.environ/COMPRESR_API_KEY
      default_on: true
```

Step 3 — start the proxy. The native registration for the Compresr guardrail is pending in upstream LiteLLM, so for now *pick one* of the two startup paths below. Both load the YAML above and register the guardrail in the proxy — choose whichever fits your deployment.

Option A — bundled `compresr-litellm` CLI (recommended). A drop-in replacement for the `litellm` command. Nothing is written into your `litellm` install, so it's the safest choice for shared or managed environments and the simplest to roll back.

```
compresr-litellm --config proxy_config.yaml --port 4000
```

Option B — stock `litellm` CLI with a one-time shim. Run `install-compresr-shim` once to register the guardrail with your active `litellm` install, then use the `litellm` command you already have wired up in Docker, systemd, or your process manager.

```
install-compresr-shim                  # one-time
litellm --config proxy_config.yaml --port 4000
```

Either way, any client pointing at `http://localhost:4000` now runs through Compresr. When the upstream LiteLLM PR lands, you'll be able to drop Step 3's extra command and just use `litellm` directly — your YAML stays unchanged.

3 Make a request

Point any OpenAI-compatible client at the proxy and send a request with a chunky tool output. Look at the response headers and the metadata block to confirm savings.

```
curl -i http://localhost:4000/v1/chat/completions \
  -H "Authorization: Bearer sk-local" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-5-mini",
    "messages": [
      {"role": "user", "content": "What did the customer say about pricing?"},
      {"role": "assistant", "tool_calls": [
        {"id": "t1", "type": "function",
         "function": {"name": "search_crm", "arguments": "{}"}}]},
      {"role": "tool", "tool_call_id": "t1", "name": "search_crm",
       "content": "<~2000 chars of CRM dump>"}
    ]
  }'
```

You'll see `x-litellm-applied-guardrails: compresr` in the response headers, plus a `compresr_stats` block in the response metadata reporting tokens saved, average ratio, and how long compression took.

`sk-local` above is just LiteLLM's local proxy master key — configure your own via `LITELLM_MASTER_KEY` or the `master_key` field in `proxy_config.yaml`.

Not using a proxy? For direct in-process compression, install `compresr` and call the SDK directly — see the `compresr_core` PDF, or the LangChain / LlamaIndex integrations for framework-native interception.

4 Knobs you'll actually tune

Most teams ship with the defaults and only revisit a handful of settings when tuning for cost vs. fidelity. The table below covers the ones that matter.

Parameter	Default	What it does
`compression_model_name`	`latte_v2`	Pick your compressor. `latte_v2` is the fast variant; `latte_v1` is the original — slightly higher fidelity, slower. The proxy defaults to `latte_v2` for inline latency; the SDK and framework integrations default to `latte_v1`.
`target_compression_ratio`	`0.5`	How aggressive to compress. Values in $[0, 1]$ are the fraction of tokens removed ($0.5 \approx$ half the tokens). Values > 1 are an Nx factor ($4 \approx$ four times smaller).
`coarse`	`true`	`true` compresses at the paragraph level — faster, blockier. `false` drops to token level — finer granularity, higher latency.
`compress_tool_outputs`	`true`	Compress `tool` / `function` messages. This is where most of the savings come from in RAG and agent workloads.
`compress_system`	`false`	Opt-in. Turn on when you have long boilerplate system prompts you want trimmed for every call.
`compress_history`	`false`	Opt-in. Compresses prior user turns — useful in long multi-turn agent loops where history balloons over time.
`fail_closed`	`false`	Default is fail-open: if Compresr is unreachable, your original request is forwarded uncompressed so traffic never stops. Set to `true` to return HTTP 503 instead.

5 Per-request overrides

Any knob from the YAML can be overridden on a single request via the `metadata.guardrail_config` block — handy for A/B testing ratios, widening scope on a specific endpoint, or pinning a model variant for one call without restarting the proxy.

```
POST /v1/chat/completions
{
  "model": "gpt-5-mini",
  "messages": [
    {"role": "user", "content": "What did the customer say about pricing?"},
    {"role": "assistant", "tool_calls": [
      {"id": "t1", "type": "function",
       "function": {"name": "search_crm", "arguments": "{}"}}]},
    {"role": "tool", "tool_call_id": "t1", "name": "search_crm",
     "content": "<long CRM dump>"}
  ],
  "metadata": {
    "guardrail_config": {
      "compression_model_name": "latte_v1",
      "target_compression_ratio": 4,
      "compress_history": true
    }
  }
}
```

Overrides apply to that request only and take precedence over the YAML defaults.